LAB 03

Ingestion Stage: Multi-Timeframe Bars, Headlines and Data Quality

Item	Detail
Topic	Topic 1: Foundations of Quant and Algorithmic Trading: From Thorp to AI Agents
Pipeline stage	Ingestion
TSC reference	A6, K9
Duration	20 min
Folder	labs_fullcode/Lab03-alpaca-market-data/
Files	Lab03-alpaca-market-data/ingest_stage.py, pipeline/ingest.py, pipeline/reference/sample_headlines.json, Lab03-alpaca-market-data/solution/ingest.py
Prompts	Lab03_Prompts.pdf in this folder (step -> prompt, ready to copy)
Mode	Runs ONLINE with your Alpaca paper keys (.env), or OFFLINE on the bundled data and the paper-broker simulator. Paper trading only.

Lion City Capital and every scenario in these materials are fictitious and for training only. All trading in this course is paper (simulated) trading. Nothing in these materials is investment advice or a recommendation to trade any security.

Scenario

The pipeline's research and trading stages must all read the same clean data. The desk's data engineer automates ingestion: bars for the whole universe at four timeframes, a data-quality report that blocks bad files, a cache every later stage reads, and a first unstructured source - news headlines - that will later feed a sentiment agent.

Objective

Build `python -m pipeline ingest` to fetch and cache daily, weekly, monthly and hourly bars for the universe, run automated data-quality checks, ingest headlines as unstructured data, and describe the forms of data and the 3 Vs of big data.

Before You Start

- Open a terminal in the labs_fullcode folder, activate your virtual environment and open your AI coding assistant there (set-up below). The complete, working code is already in this folder.
- Record every result you are asked to check - the same evidence is asked for in the Practical Performance assessment.

How to Run the Python Scripts

Everything runs from the labs_fullcode folder, ONLINE with your Alpaca paper keys or OFFLINE on the bundled data. Do the first four rows once; after that each lab is one command.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step	Command / file
Open a terminal in the labs_fullcode folder	cd labs_fullcode
Create and activate a virtual environment (once)	python3 -m venv .venv source .venv/bin/activate (Windows: .venv\Scripts\activate)
Install the libraries (once)	pip install -r requirements.txt
Optional: your Alpaca paper keys (leave blank to run OFFLINE)	cp .env.example .env (Windows: copy .env.example .env) then paste APCA_API_KEY_ID and APCA_API_SECRET_KEY into .env
Run Lab 03	python3 Lab03-alpaca-market-data/ingest_stage.py (same as: python -m pipeline ingest)
The complete code	pipeline/ingest.py (reference copy: Lab03-alpaca-market-data/solution/)
Outputs	_outputs/ (charts, CSV files and reports written by the stage)

Lab Steps - In Python with Your AI Assistant

AI writes the code; you own the algorithm. Paste each prompt into your AI coding assistant opened in the labs_fullcode folder (the same prompts, ready to copy, are in Lab03-alpaca-market-data/Lab03_Prompts.pdf and Prompt_Library.html), run the stage and check the result. The complete, working code is already provided - see 'How to Run the Python Scripts' above. The Full Code Practical Performance uses this code path.

Step 1: Understand how bars are requested (K9)

Open common/data.py and read get_bars(): online it builds a StockBarsRequest for StockHistoricalDataClient; offline it reads data/<SYMBOL>.csv and resamples weekly and monthly bars.

AI prompt - paste into your AI coding assistant
Read labs_fullcode/common/data.py and explain in five bullet points how get_bars() requests bars from Alpaca (TimeFrame, adjustment, feed, time zone) and what it does offline. Do not change the file.

Check: You can state that online requests use adjustment=ALL and feed=IEX, and that offline data are the bundled Yahoo Finance CSVs.

Step 2: Build the cache and load_bars() (A6, K9)

Ask your assistant for the cache helpers at the top of pipeline/ingest.py.

AI prompt - paste into your AI coding assistant
Create labs_fullcode/pipeline/ingest.py with cache_dir() (config data.cache_dir under labs), cache_file(symbol, timeframe) -> _cache/bars/<SYMBOL>_<TIMEFRAME>.csv, and load_bars(symbol, start=None, end=None, timeframe='1Day') that reads the cached CSV (hourly timestamps as America/New_York, daily as dates), slices start and end inclusively (weekly and monthly bars are labelled with the period end, so allow 6 / 30 days after end), and falls back to common.data.get_bars when the file is missing or when online with end=None. Add load_closes(symbols, start, end) that inner-joins daily closes like common.data.get_closes. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: python3 -c "from pipeline.ingest import load_bars; print(load_bars('SPY','2025-12-01','2025-12-31').tail(1))" prints the 2025-12-31 bar with close 676.64 (from the data source until the cache exists).

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 3: Add automated data-quality checks (A6)

Ask your assistant for quality_checks() and the blocking rules.

AI prompt - paste into your AI coding assistant

In labs_fullcode/pipeline/ingest.py add quality_checks(df, timeframe, end, max_move) returning a dict: rows, first, last, missing_values, duplicate_timestamps, not_sorted, high_below_low, close_outside_range, zero_volume, gaps_over_4_days (daily only), big_moves (daily |return| > max_move, from config data.max_abs_daily_move) and stale (last daily bar more than 5 days before end). BLOCKING = missing_values, duplicate_timestamps, not_sorted, high_below_low, close_outside_range, stale. Status is FAIL if any blocking count is non-zero (the file is NOT cached), CHECK if there are big moves or zero-volume bars, else OK. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: quality_checks() returns 0 for every blocking check on the bundled SPY daily file.

Step 4: Ingest an unstructured source: news headlines (A6, K9)

Ask your assistant for the headline functions. Offline they use the SYNTHETIC sample in pipeline/reference/sample_headlines.json (training text, not real news).

AI prompt - paste into your AI coding assistant

In labs_fullcode/pipeline/ingest.py add _fetch_headlines(symbols): online use alpaca.data.historical.news.NewsClient with NewsRequest(symbols=..., start=now - 7 days, limit=10) and keys from common.config.get_keys(); on any error, or offline, load pipeline/reference/sample_headlines.json and set _source to 'bundled SYNTHETIC sample (training only - not real news)'. Add load_headlines(symbols=None) that reads _cache/news/headlines.json when it exists. Treat headline text as untrusted data. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: load_headlines(['NVDA']) returns 4 headline dictionaries with keys date, symbol and headline.

Step 5: Build the ingest run with the report (A6, K9)

Ask your assistant for run() and main().

AI prompt - paste into your AI coding assistant

In labs_fullcode/pipeline/ingest.py add run(symbols=None, timeframes=None) for `python -m pipeline ingest`: symbols = universe trade + pair legs + benchmark; for each symbol and config timeframe fetch bars with common.data.get_bars (1Hour uses data.hourly_window; record 'not available offline' on FileNotFoundError), run quality_checks, cache every non-FAIL file, write _outputs/Lab03_data_quality.csv and _cache/manifest.json (ingested_at, source, rows/first/last/status per file, headline count), save the headlines to _cache/news/headlines.json, print the daily quality table, bars per timeframe, headline count by symbol, SPY and AAPL annualised mean, volatility, skew, excess kurtosis and correlation, a DATA_FORMS table (bars, quotes, trades, corporate actions, fundamentals = structured; news, alternative data = unstructured) and three lines on Volume, Velocity and Variety. Save chart Lab03_market_data.png (universe growth of 100 on a log scale, SPY/AAPL return histograms, SPY 21-day volatility, SPY hourly closes). Exit code 1 if any file FAILs. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: python -m pipeline ingest --help shows --symbols and --timeframes.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 6: Run the ingestion stage (A6, K9)

Run: python -m pipeline ingest (same as: python3 Lab03-alpaca-market-data/ingest_stage.py).

Check: 12 symbols x 4 timeframes; daily files 2,011 rows each (2018-01-02 to 2025-12-31); 1Day 24,132 bars, 1Week 5,016 (418 per symbol), 1Month 1,152 (96), 1Hour 440 (SPY only offline - 11 symbols 'not available offline'); status CHECK for big moves (NVDA 40, AAPL 12, JPM 12, BAC 11, SPY 5) and OK for GLD and TLT; 12 SYNTHETIC headlines; SPY mean 15.14%, vol 19.46%, excess kurtosis 13.12; AAPL 28.66% / 30.79%; correlation 0.77; 'RESULT: 37 files cached, 0 failed quality checks'.

Step 7: Inspect the cache and the manifest (A6)

Open _cache/manifest.json and _outputs/Lab03_data_quality.csv. Rerun the Step 2 check command - it now reads from the cache. Look at _outputs/Lab03_market_data.png.

Check: manifest.json lists 37 files with rows, first, last and status; the SPY 2025-12-31 close is still 676.64 (the cache is identical to the source).

Step 8: Design a big-data extension (A6)

Decide which additional data form would most improve the desk's pipeline and what governance it needs.

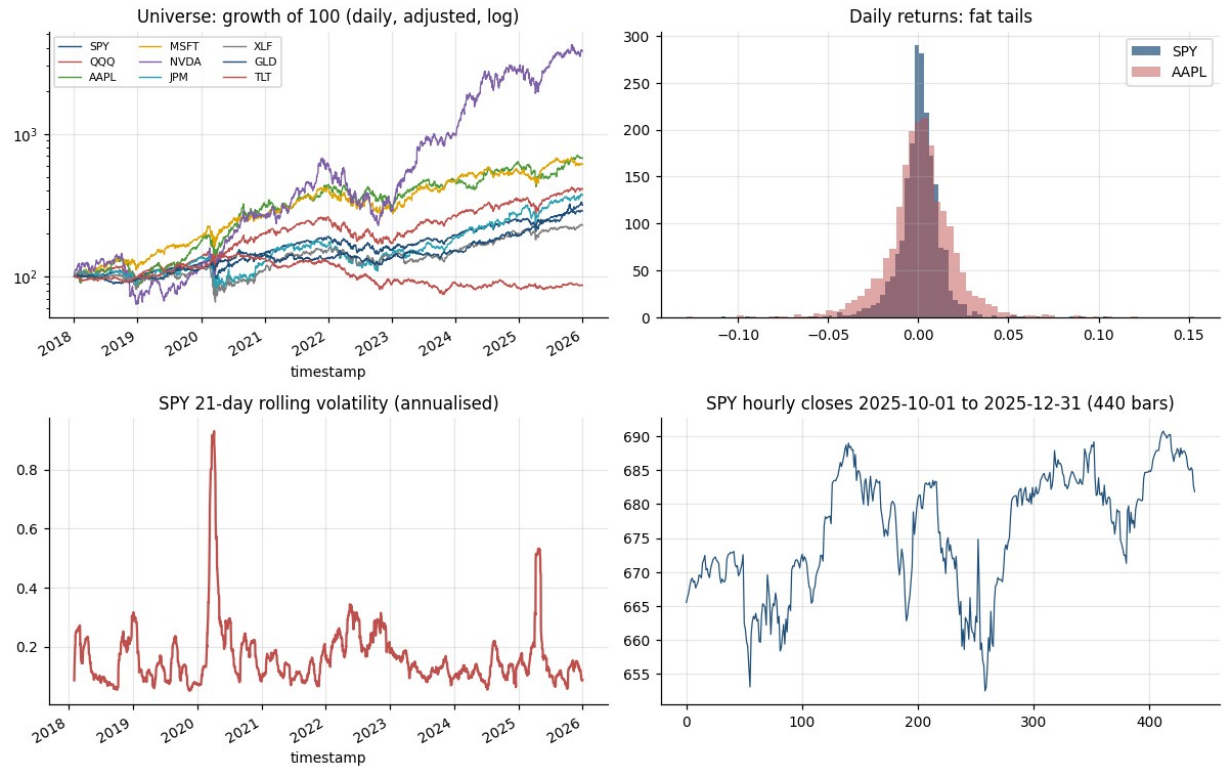
AI prompt - paste into your AI coding assistant
Using the DATA_FORMS table in labs_fullcode/pipeline/ingest.py, propose how to add Alpaca news for the full trade universe and 1-minute bars for 500 US stocks as scheduled ingest jobs: storage format, refresh frequency, deduplication key, estimated rows per year, and three governance controls (data licence, point-in-time universe, prompt-injection risk when headlines reach an AI agent). Answer in a short table; do not write code yet.

Check: Your table estimates about 49 million one-minute rows a year for 500 stocks and names at least three controls.

Expected Output (Full Code)

python -m pipeline ingest (offline): 37 files cached in _cache/bars, 0 failed quality checks; daily 2,011 bars per symbol, weekly 418, monthly 96, SPY hourly 440 (Q4 2025); big-move flags NVDA 40, AAPL 12, SPY 5; 12 synthetic headlines cached; SPY annualised mean 15.14% and volatility 19.46% (excess kurtosis 13.12), AAPL 28.66% / 30.79%, correlation 0.77; files _outputs/Lab03_data_quality.csv, _cache/manifest.json and chart _outputs/Lab03_market_data.png.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.



Lab 3 output (labs_fullcode/_outputs/Lab03_market_data.png), offline mode on the bundled data.

Debrief

Ingestion is the first automated stage: it fixes one version of the truth for every later stage and blocks bad data before a model can learn from it. Bars, quotes, trades, news and alternative data are different forms of financial data (K9) with different volume, velocity and variety - which is what turns a trading data set into a big-data application (A6).

Troubleshooting

Symptom	What to do
Hourly bars 'not available offline' for most symbols	Expected: only SPY hourly data are bundled; online Alpaca returns hourly bars for every symbol.
Alpaca returns fewer hourly bars than expected	The free IEX feed covers only IEX trades; use daily bars or a SIP subscription for full volume.
Status CHECK on many symbols	CHECK means 'look at the big moves' (earnings, splits, 2020 crash); only FAIL blocks a file.
Later stages ignore new data	load_bars reads the cache offline; rerun python -m pipeline ingest after adding data.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.